

CSCE 701 CYBERSECURITY · MODULE 13 · LECTURE NOTES

Claiming Security Honestly

What an assessment establishes.

MODULE OUTCOMES

State what a security assessment does and does not establish.

Avoid the standard overclaims.

Write a security position that is checkable.

Place this course relative to CSCE 711 and CSCE 713.

State the programme that is proportionate for a real system.

1 What an assessment is

EVERY ASSESSMENT IS A BOUNDED SEARCH, AND THE BOUND IS THE FINDING

A penetration test, a code review, a dependency scan, and a configuration audit are all searches with a time budget, a defined scope, and a particular method — and each finds what its method is capable of finding, within the time it had.

So 'no findings' means 'this method, in this time, on this scope, found nothing' — which is genuinely useful information and is not a clean bill of health (Module 01 §1's second consequence, which is the governing fact of this module).

And the scope is where the real limitation usually sits: an assessment that excluded the administrative interface, the mobile client, the third-party integration, or the build pipeline has excluded exactly where the finding was — and scope exclusions are negotiated for cost and schedule reasons rather than security ones.

So report the scope, the method, the time spent, and explicitly what was *not* examined — which makes the report a measurement rather than a verdict, and is the difference between a useful assessment and a reassuring one. A report without a stated exclusion list is incomplete regardless of its findings.

The claim	The correction
'The system is secure.'	Not a supportable statement in any form. Secure against which adversary (Module 01 §2), for which assets, under which assumptions?
'We passed the penetration test.'	A bounded search found nothing in scope in the time available — and the scope and the time are the information.

The claim	The correction
'We are compliant, therefore we are secure.'	Compliance is auditable and security is not (Module 12 §4), so budgets flow to compliance — and a fully compliant system can be insecure.
'We use encryption.'	Which data, at rest or in transit, with the keys held where and accessible to whom? (Module 07). The sentence conveys nothing without those.
'Military-grade encryption.'	Meaningless as a phrase, and misdirected as a claim — the primitive was never the weak point (Module 07 §1).
'No breaches to date.'	Entirely consistent with having no detection capability (Module 09). See the note.

The last row is the one to internalise. An organisation with no detection capability has no breaches by construction, and its clean record is evidence of nothing whatsoever — which means **the claim is strongest from organisations least able to make it and weakest from those best able to. So the useful question is never 'have you been breached' but 'how would you know', and the answer to the second tells you what the first is worth.**

2 The honest position

- **'Our threat model covers these adversaries and these assets; these threats are mitigated by these controls; these threats are accepted, for these reasons.'** The whole position, in one sentence, and it is checkable by someone else.
- **'An external assessment of this defined scope, over ten days, found three issues, all since remediated and retested; the scope excluded the mobile client and the payment integration.'** Bounded, dated, and checkable (§1).
- **'94% of employee accounts use origin-bound second factors; the remaining 6% are service accounts using scoped short-lived credentials.'** A measured, specific, high-value control (Module 12 §2).
- **'We detect injected test events at a median of four minutes and a 95th percentile of forty minutes, measured weekly.'** A tested capability rather than a counted artefact — the strongest form of security claim available.
- **And what you cannot honestly say: 'we are secure'** — unqualified, against no stated adversary, with no scope. **The first form is the one to aim for: controls mapped to threats with the acceptances stated — which is the deliverable of this entire course and of both its projects.**

3 The semester

Course	What it covers	The honest summary
CSCE 701 (this one)	Systems, operations, architecture, and people.	Where the incidents are — credentials, misconfiguration, missing authorisation, unpatched dependencies, and phishing.
CSCE 711	Cryptographic primitives, protocols, and their correct use.	The part that works, when used correctly — and the part whose failures are nearly all in key management, which is Module 07.
CSCE 713	Finding and preventing defects in code — memory safety, fuzzing, static analysis, secure development.	Where the severe bugs are — and where CSCE 627's undecidability constrains the tooling.

The division is by where the failures come from: operations and configuration first, then code defects, and only rarely the cryptography — which is the ordering the published incident data supports (Module 12's reports), and it is deliberately not the ordering a cryptography-first curriculum would imply.

4 A proportionate programme

- 1 · Threat model (Module 01) — two days of work, and it determines the value of everything after, because a control not tied to a threat is a guess.
- 2 · Authentication and authorisation (Modules 02 and 03) — origin-bound second factors and chokepoint deny-by-default authorisation. The highest-value controls available, and the two that address the leading initial-access and privilege-escalation vectors.
- 3 · Patching and inventory (Module 08) — unglamorous, consistently near the top of the initial-access data, and the inventory is what makes an advisory response possible at all.
- 4 · Logging, detection, and a tested response plan (Modules 09 and 10) — because everything above will eventually fail (Module 01 §1's last row), and the difference between a contained incident and a catastrophic one is made here.
- 5 · Then segmentation, egress filtering, platform hardening, and the web controls (Modules 04 to 06), and remove the controls people route around (Module 11 §1). Steps 1 to 4 are where the evidence says the value is, and step 5 is where most programmes start — which is the single most useful reordering in this course.

WHERE THIS COURSE LEAVES YOU

You can build a threat model with correct trust boundaries, enumerate threats systematically, map controls to them, and state explicitly what you have accepted — which is the artefact the whole course is organised around.

You can review authentication, authorisation, platform isolation, the web surface, secret handling, and the supply chain against the failures that actually occur rather than against a generic checklist — which is the practical skill, and it transfers to systems you did not design.

You can design detection that would catch a specific control failing, test that it fires, and run a response exercise that finds the gaps — and **you know that most security programmes fail operationally rather than technically** (Module 09 §4, Module 11), which is the knowledge that stops you building one that does.

The closing rule is the program's, unchanged across twenty-five courses: state what you measured, state what you assumed, and never claim more than you established. In security it means naming the adversary and the scope — because 'secure' without those is not a claim at all, and the whole of §1's table is what happens when it is made anyway.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Anderson — Security Engineering, the concluding chapters (free PDF)	§3 and §4 in context , and the best available account of why programmes are ordered the way they are rather than the way they should be.
OWASP — Application Security Verification Standard (free)	A levelled checklist to review a system against , with the levels making the proportionality of §4 explicit.
CIS Critical Security Controls (free)	§4's ordering, as a published prioritisation — and the implementation groups are a proportionality framework worth borrowing.
Verizon DBIR, the initial-access section (free, annual)	The evidence behind §4's ordering — reread it annually and check your programme against it.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Take a security claim your organisation makes publicly** and assess it against §1's table.
- 2 **Find the scope exclusions** in a past assessment report, and say what they excluded.

- 3 **Rewrite one claim** in §2's first form.
- 4 **Answer 'how would you know'** for a breach of your own system.
- 5 **Write the one-sentence position** for your system: adversaries, controls, acceptances.
- 6 **Compare your actual control spending** against §4's ordering and report the mismatch.
- 7 **Identify which step you are weakest on** and what the next action is.
- 8 **Read the current breach report's initial-access section** and check §4's ordering against it.
- 9 **Revisit what you wrote in Module 01's last exercise.**
- 10 **Project 2 is now due.** Submit the logging design, the tested detections with their predicted alert volumes, the tabletop timeline, and the blameless post-mortem.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is every assessment a bounded search, and what should a report contain?
- 2 Where is the real limitation usually found?
- 3 Give six overclaims and the correction to each.
- 4 Why is 'no breaches to date' the emptiest claim?
- 5 Give four honest claim forms and the one you cannot use.
- 6 Which form is the course's deliverable?
- 7 How do the three security courses divide, and by what principle?
- 8 Give the five-step proportionate programme.
- 9 Which steps hold the value, and where do most programmes start?
- 10 In this course, what does 'state what you assumed' mean?